IT and AI Guide

Open Sentry Beacon — one document, both halves: setting it up, and using an AI assistant to do the work. Generated from the Markdown in `docs/`*, so it cannot disagree with the repository.*

What is in here

1. Start here
2. Setting up Hope Beacon with an AI assistant
3. Email
4. Security

Start here

The one document for anyone setting up their own Hope Beacon.

This answers eight questions in order. Read it straight through if you are new. Jump to the one you need if you are not.

#	Question	Where
1	What tools does this use, and why those?	Part 1
2	What is this open-source project, and how do I take part?	Part 2
3	How do I set it up with Supabase, Vercel and Brevo?	Part 3
4	How do I put it on the internet?	Part 4
5	How do I use AI to help me?	Part 5
6	Something is broken. How do I fix it?	Part 6
7	I can code. Give me every step by hand.	Part 7
8	I cannot code. Give me the simplest path.	Part 8

You do not need to be a programmer. Part 8 is written for someone who has never opened a terminal. Part 7 is for someone who wants to know exactly what Part 8 is doing on their behalf.

Cost: all four services have a free tier, and a single church is normally well inside them. None of them asks for a card to begin. Limits change, so check each provider's own pricing page before you commit. One condition worth knowing now: **Vercel's free (Hobby) tier is for non-commercial, personal use**. A church running the app for its own members is the usual case; a business hosting it for churches, or charging for it, needs Vercel's paid tier or another host (`docs/DEPLOY-ANYWHERE.md`).

Total time: about an hour, most of it waiting for things to start.

Before any of that: try it without installing anything

Open your deployed app — or clone it and run `npm run dev` — and press **"Open the tutorial"** on the front door.

You get a complete working church with sample people in it: Guides, Explorers, a Director, conversations, prayer requests, lessons, the lot. Every feature the real app has, plus a guided walk for whichever role you pick.

The tutorial is not part of the live app, and that is the point. It invents its people in your browser. It has no account, no database and no network — it works on a plane, and nothing you type into it is sent anywhere or can reach a church's real data. It is the honest way to answer "what does this actually do?" before anyone commits to setting up a database.

You can move between the two at any time with the **LIVE / TUTORIAL** control in the header. A deployment with no database configured is the tutorial and nothing else, because there is nothing else it could be.

Part 1 — The tools, and why these ones

Four services. Each one does a job the others cannot, and each was chosen for a reason you are allowed to disagree with — this is open source, and Part 2 explains how to swap any of them.

The four

Tool	Its job in one line	Cost
GitHub	Holds the code, and how you get updates	Free for public repositories
Supabase	The database, and who is allowed to sign in	Has a free tier
Vercel	Turns the code into a website people can visit	Has a free tier
Brevo *(or any SMTP provider)*	Sends more than two invitation emails an hour	Has a free tier

Check each provider's current free-tier limits yourself before committing to one. All four are generous enough for a single church today, and all four are changed by their owners without asking us — a number printed here would be wrong eventually, and you would find out at the worst possible moment. Each one states its limits on its own pricing page.

Why Supabase

The app needs three things that normally come from three separate places: a database, a login system, and a way to say *"this Guide may read this Explorer's messages, and nobody else may."* That third one is the whole security model of a church directory, and it is the one most projects get wrong.

Supabase is PostgreSQL — a thirty-year-old database that most of the world's serious software runs on — with the login system already attached and the permission rules enforced **inside the database itself**. That last part is the reason for the choice. The rules live in `supabase/migrations/`, and they apply no matter what asks: the app, a

script, a mistake, or somebody who found an API key. A rule enforced in the app is a rule that stops applying the moment somebody talks to the database directly.

It is ordinary Postgres underneath. If you leave Supabase, the tables come with you. See BACKENDS.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/BACKENDS.md).

Why Vercel

The app is built with Next.js, which Vercel makes. Connect your GitHub repository once and every change you push becomes a live website within about a minute, with HTTPS already set up and a real address you can give to people.

The honest version: this is convenience, not necessity. Next.js runs on Netlify, on Cloudflare, on your own server. Vercel is the shortest path from "code on GitHub" to "my church can open this on their phones", and for a church volunteer doing this on a Saturday, shortest path is the right criterion. DEPLOY-ANYWHERE.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/DEPLOY-ANYWHERE.md) covers the others, and running it on your own computer.

Why Brevo

Hope Beacon is invitation-only. There is no public sign-up page — the *only* way in is a Director sending someone an invitation. So sending email is not a feature, it is the front door.

Brevo has a free tier with a daily sending allowance that a single church is unlikely to reach, and it does not ask for a card to start. Check the current number on their pricing page rather than trusting one printed here.

The important part is that Brevo is replaceable in one file. All the sending lives in a single function at the bottom of

church's own mail server are an edit there and nowhere else. Nothing else in the app knows which service you chose. EMAIL.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/EMAIL.md) has the details.

Why not "just use Google Forms and a spreadsheet"

A fair question, and for some churches the honest answer is that a spreadsheet is enough. The line is this: a spreadsheet cannot show one Guide only their own Explorers. Everyone with the link sees everything. The moment a church is recording something a person told them in confidence, the spreadsheet is the wrong tool, and no amount of care with sharing settings fixes it.

Part 2 — The project, and how to take part

What it is

Open Sentry Beacon is a free, open-source app for churches walking alongside people who want to know more about faith. One Guide, one Explorer, one conversation, and a record of it that only the right people can see.

It is licensed for you to take, change, run and keep — under the AGPL-3.0. Fork it, rename it, strip out what you do not need, charge for hosting or support. No permission required and no fee.

The one condition, in a sentence: **if you change it and run it for people over a network, you offer those people your changed source.** Running this repository unmodified for your own church obliges you to nothing at all. Keeping your changes on your laptop obliges you to nothing. The clause only bites when other people are using your modified version. The app already carries a *view the source* link in **Settings** — if you deploy a modified version, repoint that link at your own repository and you have done everything the licence asks. Full detail is in the project's README.

Your data is yours and it never comes to us. When you set up your own Supabase project, the database is yours, in your account, under your billing. Nobody involved in this project can see inside it. There is no central server, no telemetry, no "phone home".

The repository

```
app/          the screens people see
components/   the pieces those screens are built from
lib/          the logic, including lib/live/data.ts — every database call
supabase/
  migrations/ the database, as numbered SQL files. Run in order.
  functions/  server-side code, including the invitation sender
docs/         this file and its neighbours
tests/        the checks that run before anything ships
```

How to take part

You need a free GitHub account. Nothing else.

Found a bug, or want a feature? Open an *Issue*. Go to the repository, click **Issues**, then **New issue**. Describe what you expected, what happened, and what you were doing. That is a real contribution — a clear bug report is worth more than a bad fix.

Want to change something yourself?

1. **Fork** the repository (button, top right). You now have your own copy.
2. Make your change — on GitHub's own website is fine for small edits; click any file and press the pencil.
3. Click **Contribute → Open pull request**. Say what you changed and why.
4. Somebody reads it, discusses it with you, and it either goes in or it does not. Either way you will get a reason.

Nothing you do to your fork can affect anyone else. A pull request is a *request*. This is the part people are most nervous about and it is the part with the least to fear.

Before opening a pull request, run `npm run verify:all`. It runs the same checks the project runs. If it passes locally it will almost certainly pass for everyone.

What good contributions look like here

- **Say why, not just what.** The code in this repository explains its reasoning in comments, including the mistakes that led to it. Match that.

- **Do not break the demo.** The app must still run with no database at all — clone, `npm run dev`, working church with sample people. `tests/no-backend.js` enforces it.
- **Never weaken a security rule to make something work.** If a rule is in your way, say so in the pull request and let it be discussed. Rules in `supabase/migrations/` are the only thing standing between a church's private conversations and everybody.

Part 3 — Setting it up

Three accounts, in this order. Each step ends with something you can check.

Before you start

Install Node.js (https://nodejs.org) version 22 or newer. It is a normal installer — download, next, next, done.

Check it worked. Open a terminal (**Terminal** on Mac, **PowerShell** on Windows) and type:

```
node --version
```

You want a number starting with `v22` or higher. If you get "command not found", Node is not installed — go back and run the installer.

Step 1 — Get the code

```
git clone https://github.com/klydo131/open-sentry-beacon.git
cd open-sentry-beacon
npm install
npm run dev
```

Open `http://localhost:3000`.

You now have a working church app with sample people in it, running entirely in your browser with no database, no accounts and no configuration. Click around. Nothing you do here can break anything.

> **Check:** you can see a Guide's desk with Explorers on it. If you can, the hardest part of this whole document is already behind you.

Step 2 — Supabase, the database

1. Sign up at supabase.com (https://supabase.com). Free, no card.
2. **New project.** Give it a name and a database password — **save that password somewhere**, it is not recoverable.
3. Wait about two minutes while it starts.
4. Open **Settings → API** and leave that page open.

Then, back in your terminal, in the project folder:

```
npm run setup
```

It asks two questions and checks your answers.

> **It will stop you if you paste the wrong key.** That settings page shows two that look alike and sit next to each other. One is meant to go to every visitor's browser; the other bypasses every security rule you are about to create.
>
> Supabase issues them in two formats depending on when your project was made — either a long string with two dots in it, or a pair beginning
>
> way.** The setup command understands both formats and refuses the dangerous one in both. This is the single most common serious mistake, and the app will not let you make it.

Step 3 — Create the tables

One command, and it is the same command for every update later. In Supabase, open **Connect** at the top of your project and copy the connection string (the one that starts `postgresql://`). If it offers a choice, take **Session pooler**: it works on every home and church network. Put your database password where it says `[YOUR-PASSWORD]`. Then, in the project folder:

```
npx supabase db push --db-url "<the connection string>"
```

Answer **Y** when it lists the files. It runs every file in

the app later (`git pull`), run exactly the same command: it runs only the new files, and says *"Remote database is up to date"* when there are none.

If the site updates before the database does -- the push reaches Vercel in minutes, and you run this command when you get to it -- the app keeps working. A newer feature whose tables are not there yet is simply not offered: the chat, for example, keeps its messages, photos and live refresh, and Reply, reactions and voice messages appear once this command has run. Nothing is lost either way.

> **One line it prints is expected:** *"Skipping migration 0001a_fix_policy_recursion.sql"*. That early fix was later rewritten by the files after it, so the database is the same with or without it. That is not a promise: every change to this repository builds the database both ways and fails unless the two are identical (`.github/workflows/fresh-install.yml`).
>
> **If it cannot connect** and your password has symbols such as `@`, `#` or
>
> **Settings → Database** to letters and numbers only, and copy the string again.

Without a terminal, the same result by hand: open the **SQL Editor**, then open each file in `supabase/migrations/` **in filename order**, paste it in and click **Run**. There are well over a hundred. Filename order means character by character: `0001_core_schema.sql` comes before `0001a_fix_policy_recursion.sql`, the numbered files `0001` to `0049` come first, and the dated ones (`2026…`) follow in date order. The first fifteen, and what each is for:

```
0001_core_schema.sql               the tables, and the security rules
0001a_fix_policy_recursion.sql     a fix to those rules — must follow 0001
0002_invitations.sql               invitations
0003_invite_approval_gate.sql      who may invite whom
0004_live_api_permissions.sql      what the browser may call
0005_platform_function_acl.sql     locking down internal functions
0006_blog.sql                      Guides' blog posts
0007_prayer.sql                    prayer requests
0008_library.sql                   shared study material
0009_meetings.sql                  scheduling
0010_lock_definer_functions.sql    a security fix — see note below
0011_ministry.sql                  recommendations and follow-ups
0012_lessons_and_notifications.sql lessons and the notification bell
0013_the_invitation_is_the_approval.sql  the sign-up form, and approval on invite
20260816130240_approval_revocation_gate.sql  removing approval takes effect at once
…and every file after it, to the newest.
```

Order is not optional — each file builds on the one before. By hand you must also remember which files you have run: the next update adds files, and only those are to be run. The command above keeps that list for you.

> **Why 0010 exists, because it is worth your time.** An earlier migration tried to take away permission to run certain internal database functions and used
>
> to `PUBLIC`, and revoking from one member of a group does not remove what the group has. Four rounds of security testing missed it, because they all tested *tables* and this was a *function*. Supabase's own linter found it. If you ever write `revoke`, check what `PUBLIC` holds.

Restart the app: stop it with `Ctrl+C`, then `npm run dev` again.

> **Check:** open `http://localhost:3000/setup`. That page tests your work and tells you which step you are actually on rather than making you guess.

Step 4 — Make yourself the first Director

Somebody has to be first, and they cannot be invited, because there is nobody to invite them. So the first account is made by hand, in SQL — deliberately, since the alternative is a page on the internet that hands out Executive Director to whoever finds it.

Sign up in the app with your own email. Then, in the Supabase SQL Editor, run

inside it with yours.

> **Check:** sign in. You should land on the Director's screen.

Step 5 — Brevo, so invitations can be sent

1. Sign up at brevo.com (https://www.brevo.com). Free, no card.
2. Verify a sender address: **Senders, Domains & Dedicated IPs → Senders → Add a sender**. Brevo will not send from an address it has not confirmed is yours.
3. Create an API key: **SMTP & API → API Keys → Generate a new API key**. Copy it — it starts `xkeysib-` and is shown once.

> **Turn the IP restriction OFF for this key.** Brevo can limit a key to certain IP addresses. The code that sends your invitations runs on Supabase's servers, which have no fixed address, so an IP restriction blocks every invitation you ever send. The error message it produces says the credentials were rejected, which sends you looking at the key instead of the restriction.

Now give those to Supabase. In your Supabase project: **Edge Functions → Secrets**, and add three:

Name	Value
`BREVO_API_KEY`	the `xkeysib-…` key
`BREVO_SENDER`	the sender address you just verified
`SITE_URL`	your app's address, e.g. `https://your-church.vercel.app`

your address.

Then deploy the invitation sender:

```
npx supabase login
npx supabase functions deploy invite --project-ref YOUR_PROJECT_REF
```

rejected. Your project ref is the part of your Supabase URL before

No terminal handy? Paste it in the dashboard instead: **Edge Functions → Deploy a new function**, name it exactly `invite`, and paste the contents of

> **Check, and do not skip this one:** invite somebody — use a second email address of your own. The message should arrive **from your church's name, not from Supabase**, and the link in it should open your app and not
>
> link to copy instead of sending it, the three secrets above are not set correctly. That is the single most common setup failure.

Step 6 — Push notifications to phones *(optional, 15 minutes)*

Without this the app still shows notifications while it is open. With it, a message reaches a phone that is locked. On an iPhone that only ever works for the app added to the Home Screen, on iOS 16.4 or later -- Apple's rule, not ours.

1. Make a key pair, once: `npx web-push generate-vapid-keys`. Keep the private one private.

2. In Supabase, **Edge Functions → Secrets**: `VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY`, and `VAPID_SUBJECT` (a `mailto:` address of yours).
3. Deploy the sender: `npx supabase functions deploy notify --project-ref YOUR_PROJECT_REF`
4. In Vercel, add `NEXT_PUBLIC_VAPID_PUBLIC_KEY` with the **public** key, and redeploy.
5. Tell the database where the sender is. In the Supabase **SQL editor**:

```sql
select vault.create_secret('<your service role key>', 'notify_service_key');
select vault.create_secret('https://YOUR_PROJECT_REF.supabase.co/functions/v1/notify', 'notify_function_url');
```

These live in your project's own Vault. Never put either value in the repository or in anything named `NEXT_PUBLIC_`.

Until step 5 is done nothing is sent and nothing breaks: notifications are saved exactly as before.

Step 7 — Lock the invitation sender to your site *(2 minutes)*

In **Edge Functions → Secrets**, add `BEACON_ALLOWED_ORIGINS` with your site's address, for example `https://your-church.vercel.app` (several can be listed, separated by commas). Without it the invitation sender answers any website that calls it -- it still refuses anybody who is not one of your Directors, but there is no reason to let other sites read its replies. The place search in Step 7b reads the same setting.

Step 7b — Place suggestions for meet-ups *(2 minutes, optional)*

When two people arrange to meet in person, the place box suggests places as they type -- "jollibee imus" shows each branch with its street and barangay -- and tapping one pins that exact spot. It uses Photon, a free place search over OpenStreetMap run by komoot; there is no account to make and nothing to pay. komoot offers it free **for fair use**: one church typing a meet-up place now and then is exactly that; a company hosting many churches on it is not.

1. Deploy it: `npx supabase functions deploy places --project-ref YOUR_PROJECT_REF`
2. *(Optional)* In **Edge Functions → Secrets**, add `PLACES_NEAR` with a point near your church, as `latitude,longitude` (for example `14.4,120.9`), so nearby places come first for people who have not pinned a place yet.
3. *(Recommended)* Add `PLACES_USER_AGENT` with your app's name and a contact address, for example `Grace Chapel app (admin@yourchurch.org)`. Photon asks callers to say who they are. Without it every copy of this app sends the same name, and one heavy user could get that name blocked for all of them.
4. *(Only for heavy use)* If you host many churches, run your own Photon and put its `https://` address in `PLACES_PHOTON_URL`.

Only the words typed and a town-level point leave your server; the privacy notice already says so. Without this step the place box is an ordinary box: typing an address or pasting a map link works exactly as before.

Step 8 — Check your copy is the same app *(5 minutes, optional)*

Everything in `supabase/migrations/` builds the same database that the first church runs, whether you used the one command or ran every file by hand; that is checked on every change by `.github/workflows/fresh-install.yml`, which builds it both ways and compares them. To see it for yourself, run

the structure, never anybody's rows, and prints one line per kind of thing -- tables, columns, rules, functions. The same file run on any correct install prints the same lines.

Part 4 — Deploying

Getting it onto the internet, where your congregation can reach it.

1. **Push your copy to your own GitHub repository.** If you forked, this is already done.
2. Sign in at vercel.com (https://vercel.com) **with GitHub**.
3. **Add New → Project**, pick your repository, click **Import**.
4. Before deploying, open **Environment Variables** and add the same two values `npm run setup` wrote into your `.env.local`:

| Name | Value | |---|---| | `NEXT_PUBLIC_SUPABASE_URL` | `https://<your-ref>.supabase.co` | | `NEXT_PUBLIC_SUPABASE_ANON_KEY` | the publishable key (`eyJ…` or `sb_publishable_…`) |

> **Not the service_role key. Ever.** Anything named `NEXT_PUBLIC_` is sent > to every visitor's browser. That is fine for the publishable key — it is > designed for it, and your security rules are what protect your data. The > service_role key bypasses every one of those rules.

1. **Deploy.** About a minute.
2. Copy your new address and set it as `SITE_URL` in Supabase's Edge Function secrets (Part 3, Step 5), then redeploy the invite function.
3. In Supabase, **Authentication → URL Configuration**:

| Setting | Value | The mistake to avoid | |---|---|---| | **Site URL** | `https://your-address` | **Not** `.../join`. This is the base for password resets too, and pointing it at one page breaks the others. | | **Redirect URLs** | `https://your-address/**` | Leaving this empty is the failure below. |

> **This is the setting that silently ruins invitations.** If your address is > not in the Redirect URLs list, Supabase ignores the redirect the app asks > for and mails whatever Site URL says — and a project still on its defaults > mails everybody a link to `http://localhost:3000`, a machine they do not > have. The send reports success. Nobody can tell from inside the app. > > Hope Beacon works around this by also showing the Director a working link > on screen after every invitation, built from `SITE_URL` rather than from > the dashboard. Set these two anyway.

From now on, every push to your `main` branch updates the live site automatically. There is no second step and no "publish" button.

> **Check:** open your address on your phone, on mobile data with wifi off. Sign in. If it works there, it works.

Before the first real member is invited, work through GOING-LIVE.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/GOING-LIVE.md): a clean project, ten steps each with its proof, and `supabase/tests/ready-for-real-members.sql` to confirm them.

Part 5 — Using AI to do the work

You can hand most of this document to an AI assistant and have it do the typing. This section is about doing that *well*, because the failure modes are specific.

What to use

Any of Claude, ChatGPT, Gemini, GitHub Copilot, Cursor. The free tiers are enough for setup. There is a longer list in AI-TOOLKIT.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/AI-TOOLKIT.md).

The one thing that matters most

Give it the repository, not a description of the repository.

An AI that has read `supabase/migrations/0001_core_schema.sql` knows your exact security rules. One that has only been told "it's a church app with Supabase" will invent a plausible schema that does not match yours, and everything it writes afterwards will be subtly wrong in a way that is hard to see.

Tools like Claude Code, Cursor and Copilot Workspace read the actual files. Use one of those if you can. If you are using a chat window, paste the real file.

Prompts that work

Setting up:

> I am setting up Open Sentry Beacon from `github.com/klydo131/open-sentry-beacon`. Read `docs/START-HERE.md` and `docs/SETUP.md`. I am on Part 3 Step 3 and the SQL editor returned this error: `<paste the exact error>`. What went wrong and what do I run instead?

Making a change:

> In this repository, I want Explorers to be able to mark a lesson as finished. Before writing anything: read `supabase/migrations/0012_lessons_and_notifications.sql` and `lib/live/data.ts`, and tell me which files you would change and why. Do not write code yet.

That last sentence is the important one. Ask for the plan first. It takes ten seconds to read a plan and a long time to unpick a confident wrong change.

Understanding something:

> Explain what `supabase/migrations/0003_invite_approval_gate.sql` does, in plain English, as if I do not know SQL. What could go wrong if I removed it?

The rules, and they are not optional

Never paste a key into a chat window. Not the service_role key, not the Brevo key, not your database password. Say `<my API key>` instead. If you have already pasted one, go and rotate it now — it is a button in the same place you created it.

Make it show you, not tell you. "Did that work?" gets you a cheerful yes. "Run the query that proves it and show me the output" gets you the truth. This document exists partly because an AI reported a fixed invitation system twice before anyone clicked a link.

Ask for the negative test. If it says a security rule works, ask it to prove the rule *blocks* what it should block. A test where nothing fails has proved nothing. Every security fix in this project is checked that way, and the reason is that a test suite once passed while refusing four things for a completely unrelated reason.

It will be confidently wrong sometimes. Not occasionally — regularly, and most convincingly on the things it half-knows. Check anything that touches money, security, or somebody's private conversation.

Part 6 — When something breaks

Ordered by how often they actually happen.

"The invitation email never arrives"

Start here, because it is the answer four times out of five: Supabase's built-in mailer sends TWO messages an hour. For the whole project. Not per person.

That number is measured, not estimated — a real church's auth log shows at most two successes in any hour and `over_email_send_rate_limit` on everything after, including invitations the Director believed had gone out. A Director inviting four people in one sitting has two of them silently refused.

It **cannot be raised** while the built-in mailer is in use. It is not a setting in your dashboard. Your options are:

1. **Send the link by hand.** Every invitation shows the Director a working one-time link on screen, whether the email went or not. WhatsApp it. This works today, with no setup, and has no limit.
2. **Connect your own SMTP provider** — the section below. Thirty an hour by default, and configurable upwards.

The app now tells you which failure it was, in words, instead of showing the provider's raw text. Four different refusals arrive looking similar:

What you see	What it means	What to do
"wait N seconds"	One message per address per minute. The first one **did** send.	Wait, press Send once.
"used up its email for the hour"	The project's two-an-hour quota. **Nothing sent.**	Send the link by hand, or connect SMTP.
"That is IP blocking"	Your provider is refusing the connection's address.	Turn the blocking off — see below.
Anything else	Your provider refused and said why.	Usually a regenerated key or an unverified sender.

Connecting Brevo (or any provider) over SMTP

This is a dashboard form, not code. Nothing in the app changes — same function, same links, and no API key stored in your project.

In Brevo, first: Settings → Security → Authorized IPs → **Deactivate blocking**.

> **Do not add an IP address instead.** This is the single most expensive mistake in this project's history, and it was made twice. The connection is made by your *mail service's* servers, not by the computer you are sitting at, and those addresses change between sends. Adding your own IP authorises the one machine that will never connect. Brevo applies this list to SMTP keys as well as API keys — an unrecognised address gets

Then: Transactional → Settings → SMTP relay → **Generate a new SMTP key**. Copy the login (it looks like `something@smtp-brevo.com`) and the key. **An SMTP key is not an API key** — the API key will not authenticate here.

Then in Supabase, Project Settings → Authentication → SMTP Settings:

Field	Value
Host	`smtp-relay.brevo.com`
Port	`587`
Username	the SMTP login
Password	the SMTP **key**
Sender email	an address you have verified in Brevo

Last: Authentication → Rate Limits → raise **emails sent per hour** from 2.

"Someone I invited shows as joined, but they never signed up"

Fixed, and worth knowing why, because the same confusion bites other apps.

Sending an invitation **creates the person's account before the message goes**. So neither "an account exists" nor "the link was opened" means anybody arrived — and opening the link is something *you* do when you test it. Hope Beacon now counts somebody as joined only when they have finished the form and chosen their own password. Everyone else stays under **Waiting**, with a **Re-send** button.

> **Do not open an invitation link yourself to check it works.** Opening it signs you out and starts that person's sign-up on your device — the link is single-use, so you also consume it. The app now stops and asks before doing that, but the habit is the fix.

Read the actual error rather than guessing: Supabase → **Edge Functions → invite → Logs**.

"The invitation link says invalid or expired"

If you are running a copy from before 18 August 2026, update it. There was a real bug: the link carried the wrong one of the three tokens Supabase issues, so *every* invitation failed on arrival for everyone, and the message blamed expiry. Pull the latest `main`.

If you are up to date, check `SITE_URL` matches your real address, and that

Configuration → Redirect URLs**.

Note that invitation links genuinely do work only once. If you clicked it to test, the next click will fail correctly.

"I signed in and it sent me straight back to the sign-in page"

Your account exists but is not approved. Somebody with a Director account must approve you — or, if you are the first person, you have not run

"It works on my computer but the deployed site is blank"

The environment variables are missing on Vercel. `.env.local` is on your machine and deliberately never uploaded. Add them in Vercel's project settings (Part 4, Step 4) and redeploy — **changing them does not redeploy by itself.**

"Permission denied" or "row violates row-level security"

The security rules are working; the account asking does not have the right. This is the system doing its job, not a bug. Check the account's role and that it is approved. **Do not "fix" this by turning off row level security** — that makes every private conversation in your church readable by anyone with your publishable key, which is in every visitor's browser.

"Migration failed"

Read the error; Postgres is unusually specific. The two common causes are running files out of order, or running one twice. Most are written to be safe to re-run; if one is not, the error will say the object already exists, which is harmless.

never runs one twice. If you started by hand and switch to it, it will try to run everything again, because it has no record of what you ran; finish by hand in that case, or start again on a fresh project.

Nothing above matches

1. `npm run verify:all` — it checks a lot and names what it finds.
2. Supabase → **Logs** for database errors, **Edge Functions** → **Logs** for email.
3. Your browser's console (`F12`) for anything on screen.
4. Open an Issue on GitHub with the exact error text. Somebody has probably hit it.

Part 7 — The manual path, for people who code

Everything Part 8 does for you, done by hand. Assumes you are comfortable with a terminal, git, and SQL.

```
# 1. Code
git clone https://github.com/klydo131/open-sentry-beacon.git
cd open-sentry-beacon && npm install

# 2. Runs with no backend at all — this is the design, not a fallback
npm run dev            # localhost:3000, sample data, in-browser only

# 3. Point it at your own Supabase project
cat > .env.local <<'EOF'
NEXT_PUBLIC_SUPABASE_URL=https://<ref>.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=<publishable key: eyJ... or sb_publishable_...>
EOF
# or: npm run setup  — same result, and it refuses the service_role key

# 4. Schema. The one command, which also remembers what it ran:
npx supabase db push --db-url "$DATABASE_URL"
#    ...or every file by hand, in BYTE order. Without LC_ALL=C most Linux
#    machines sort 0001a_ before 0001_ (the language setting ignores the
#    underscore), and the loop would run a fix before the thing it fixes.
export LC_ALL=C
for f in supabase/migrations/*.sql; do
  psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f "$f" || { echo "Stopped at $f"; break; }
done

# 5. Bootstrap yourself, after signing up in the app with this address
psql "$DATABASE_URL" -f supabase/seed/01_make_me_the_first_director.sql

# 6. Email
npx supabase secrets set \
  BREVO_API_KEY=xkeysib-... \
  BREVO_SENDER=invites@yourchurch.org \
  SITE_URL=https://yourchurch.vercel.app \
  --project-ref <ref>
npx supabase functions deploy invite --project-ref <ref>
npx supabase functions deploy places --project-ref <ref>   # place suggestions (optional)

# 7. Prove it before you trust it
npm run verify:all
npm run build
```

What to read, in this order

File	Why
`lib/mode.ts`	`IS_LIVE` / `IS_DEMO`. The whole switch is the presence of the two keys.
`lib/live/data.ts`	Every database call the browser makes. One file on purpose.
`supabase/migrations/0001_core_schema.sql`	Tables and the security rules. Read the comments.
`supabase/functions/invite/index.ts`	The only code holding the service_role key.
`app/api/auth/sign-in/route.ts`	Sign-in runs server-side so credentials never touch client code.

Things that will bite you

- **Contracts and migrations move forward only.** Never edit an applied migration; add a new numbered one. Somebody else's database already ran the old one.
- `revoke ... from anon` **may do nothing.** If the grant went to `PUBLIC`, revoking from one role changes nothing. See `0010_lock_definer_functions.sql` and check `PUBLIC` first.
- `?code=` **is not a generic word for "token".** In `@supabase/supabase-js` it selects the PKCE route, which needs a verifier the browser wrote when it *started* the flow. A server-minted link has none, so it fails on every device. Emailed links must carry `token_hash` and redeem with `verifyOtp`.
- **A CHECK constraint may only call IMMUTABLE functions.** `current_date` is not one. Postgres refuses it, and it is right to.
- **Deploying an Edge Function is not committing it.** They are separate actions and this project has twice had a function running live with no source in the repository.
- `tests/no-backend.js` **is a real constraint.** The app must still run with no database. Adding a hard requirement on an env var breaks the build.

Part 8 — The simple path, for everyone else

You have never used a terminal. That is fine. Follow this exactly.

Every step tells you what you should see. If you do not see it, stop there — the step after will not fix it — and check Part 6.

What you will have at the end

A website at your own address that your church signs into on their phones.

Step 1 — Install Node.js *(5 minutes)*

Go to nodejs.org (https://nodejs.org) and download the big green **LTS** button. Run the installer, click next until it finishes.

Step 2 — Open a terminal *(1 minute)*

- **Windows:** Start menu, type `powershell`, press Enter.
- **Mac:** `Cmd+Space`, type `terminal`, press Enter.

A window with text appears. You type commands here and press Enter. It cannot break your computer.

Type this and press Enter:

```
node --version
```

> **You should see** something like `v22.14.0`. If it says "not recognized", Node did not install — go back to Step 1.

Step 3 — Download the app *(3 minutes)*

Type these one at a time, pressing Enter after each and waiting for it to finish:

```
git clone https://github.com/klydo131/open-sentry-beacon.git
```

```
cd open-sentry-beacon
```

```
npm install
```

The last one prints a lot and takes a minute or two. Warnings are normal.

Step 4 — See it working *(1 minute)*

```
npm run dev
```

Open your web browser and go to `http://localhost:3000`.

> **You should see** Hope Beacon with sample people in it. **This is the whole app.** Click around — it is not connected to anything and you cannot break it.

Leave this window running. To stop it later, press `Ctrl+C`.

Step 5 — Make a database *(10 minutes, mostly waiting)*

1. Go to supabase.com (https://supabase.com), **Start your project**, sign in with GitHub.
2. **New project.** Name it after your church. Set a database password and **write it down** — you cannot get it back.
3. It takes about two minutes to start. Wait.
4. Click **Settings** (gear, bottom left) → **API**. Leave this page open.

Step 6 — Connect them *(3 minutes)*

Open a **second** terminal window (leave the first running). Then:

```
cd open-sentry-beacon
```

```
npm run setup
```

It asks two questions. Copy the answers from the Supabase page you left open.

> **If it says you pasted the wrong key, it is right and you did.** That page shows two long keys that look almost identical. You want the **publishable** or **anon** one, not the one labelled `service_role`. This check exists because that mistake would expose everything.

Step 7 — Build the tables *(15 minutes)*

This is the longest step. It is repetitive, not hard.

In Supabase, click **SQL Editor** in the left sidebar.

On your computer, open the `open-sentry-beacon` folder, then `supabase`, then

For each file, in order, top to bottom:

1. Open it (any text editor — Notepad, TextEdit).
2. Select all (`Ctrl+A` / `Cmd+A`), copy (`Ctrl+C` / `Cmd+C`).
3. In Supabase's SQL Editor, paste, click **Run**.
4. Wait for "Success". Then the next file.

> **The order matters and is not optional.** Each file builds on the one before.

> **You should see** "Success. No rows returned" for most of them. That is what success looks like here.

Step 8 — Restart *(1 minute)*

Go back to the first terminal window. Press `Ctrl+C`. Then:

```
npm run dev
```

> **Check:** go to `http://localhost:3000/setup`. That page tests everything you just did and tells you if anything is missing.

Step 9 — Make yourself the Director *(5 minutes)*

At `http://localhost:3000`, sign up with your real email and a password.

Then open `supabase/seed/01_make_me_the_first_director.sql`. Find the email address inside and replace it with yours. Copy the whole file into Supabase's SQL Editor and click **Run**.

Sign out of the app and sign back in.

> **You should see** the Director's screen, with the whole church on it.

Step 10 — Put it on the internet *(10 minutes)*

1. Go to vercel.com (https://vercel.com) and sign in **with GitHub**.
2. **Add New → Project.** Find `open-sentry-beacon` and click **Import**.
3. Before clicking Deploy, expand **Environment Variables** and add two. They are in the `.env.local` file that Step 6 created in your project folder — open it and copy both lines.
4. Click **Deploy** and wait about a minute.
5. It gives you an address like `open-sentry-beacon-xyz.vercel.app`. **That is your app.** Open it on your phone.

Step 11 — Turn on invitations *(10 minutes)*

Nobody else can join until this is done.

1. Sign up at brevo.com (https://www.brevo.com) — free, no card.
2. **Senders, Domains & Dedicated IPs → Senders → Add a sender.** Use a church address. Confirm it from the email they send you.
3. **SMTP & API → API Keys → Generate a new API key.** Copy it now; it is shown once. **If it offers to restrict the key to an IP address, say no** — it will block every invitation you send.
4. In Supabase: **Edge Functions → Secrets**, add three:

- `BREVO_API_KEY` — the key you just copied
- `BREVO_SENDER` — the sender address you verified
- `SITE_URL` — your Vercel address, starting `https://`

1. In Supabase: **Authentication → URL Configuration**. Set **Site URL** to your Vercel address, and add `https://your-address/join` under **Redirect URLs**.

> **Check, and actually do it:** in your app, invite a second email address of your own. The message should arrive **from your church**, and its link should open your app. If it does, you are finished — invite your Directors and Guides, and they invite everyone else.

You are done

Keep somewhere safe: your Supabase database password, your Brevo API key, and your app's address.

To update later: in GitHub, open your fork, click **Sync fork**. Vercel rebuilds automatically. If an update adds a migration, run it the way you did in Step 7.

Where to go next

Document	What it covers
SETUP.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/SETUP.md)	The short version of Part 3
BUILD-YOUR-OWN.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/BUILD-YOUR-OWN.md)	Why the backend is shaped this way
IN-ANY-LANGUAGE.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/IN-ANY-LANGUAGE.md)	Your own app or server in another programming language, with working examples; putting the screens into other languages
BACKENDS.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/BACKENDS.md)	Using something other than Supabase
DEPLOY-ANYWHERE.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/DEPLOY-ANYWHERE.md)	Hosting somewhere other than Vercel, or on your own computer
PLATFORMS.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/PLATFORMS.md)	Which phones, tablets and browsers it is tested on
EMAIL.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/EMAIL.md)	Using something other than Brevo
BACKEND-MAP.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/BACKEND-MAP.md)	Every table, who may read it, and where the rule lives
SECURITY.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/SECURITY.md)	The rules, and how to check them yourself
ONBOARDING.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/ONBOARDING.md)	Getting a real church using it
AI-TOOLKIT.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/AI-TOOLKIT.md)	Longer version of Part 5
UPDATES.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/UPDATES.md)	Keeping your copy current

Stuck? Open an Issue at

what you expected, and the exact error. That is a contribution too.

Setting up Hope Beacon with an AI assistant

For IT people who would rather describe the job than type it.

Everything in START-HERE.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/START-HERE.md) can be handed to an AI assistant. This document is about doing that *well*, because the ways it goes wrong are specific and predictable.

Read START-HERE.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/START-HERE.md) first — this assumes you know what is being set up and why.

You can hand most of this document to an AI assistant and have it do the typing. This section is about doing that *well*, because the failure modes are specific.

What to use

Any of Claude, ChatGPT, Gemini, GitHub Copilot, Cursor. The free tiers are enough for setup. There is a longer list in AI-TOOLKIT.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/AI-TOOLKIT.md).

The one thing that matters most

Give it the repository, not a description of the repository.

An AI that has read `supabase/migrations/0001_core_schema.sql` knows your exact security rules. One that has only been told "it's a church app with Supabase" will invent a plausible schema that does not match yours, and everything it writes afterwards will be subtly wrong in a way that is hard to see.

Tools like Claude Code, Cursor and Copilot Workspace read the actual files. Use one of those if you can. If you are using a chat window, paste the real file.

Prompts that work

Setting up:

> I am setting up Open Sentry Beacon from `github.com/klydo131/open-sentry-beacon`. Read `docs/START-HERE.md` and `docs/SETUP.md`. I am on Part 3 Step 3 and the SQL editor returned this error: `<paste the exact error>`. What went wrong and what do I run instead?

Making a change:

> In this repository, I want Explorers to be able to mark a lesson as finished. Before writing anything: read `supabase/migrations/0012_lessons_and_notifications.sql` and `lib/live/data.ts`, and tell me which files you would change and why. Do not write code yet.

That last sentence is the important one. Ask for the plan first. It takes ten seconds to read a plan and a long time to unpick a confident wrong change.

Understanding something:

> Explain what `supabase/migrations/0003_invite_approval_gate.sql` does, in plain English, as if I do not know SQL. What could go wrong if I removed it?

The rules, and they are not optional

Never paste a key into a chat window. Not the service_role key, not the Brevo key, not your database password. Say `<my API key>` instead. If you have already pasted one, go and rotate it now — it is a button in the same place you created it.

Make it show you, not tell you. "Did that work?" gets you a cheerful yes. "Run the query that proves it and show me the output" gets you the truth. This document exists partly because an AI reported a fixed invitation system twice before anyone clicked a link.

Ask for the negative test. If it says a security rule works, ask it to prove the rule *blocks* what it should block. A test where nothing fails has proved nothing. Every security fix in this project is checked that way, and the reason is that a test suite once passed while refusing four things for a completely unrelated reason.

It will be confidently wrong sometimes. Not occasionally — regularly, and most convincingly on the things it half-knows. Check anything that touches money, security, or somebody's private conversation.

A working session, start to finish

Copy this into an assistant that can read files (Claude Code, Cursor, Copilot Workspace), one message at a time. Wait for each answer.

1. Orient it

> Clone `https://github.com/klydo131/open-sentry-beacon` and read
>
> what the four services are for, and what order I have to do things in. Do not change anything yet.

If the summary is wrong, stop. Everything after this depends on it.

2. Get it running locally

> Get the app running on my machine with no database, the way the README describes. Tell me the exact commands and what I should see.

3. The database

> I have created a Supabase project. Walk me through applying every migration in `supabase/migrations/` in order. For each file, tell me in one line what it adds, so I know what I am running. Warn me about anything irreversible.

4. Prove the security rules — do not skip this

> Write me SQL that proves a Guide cannot read another Guide's Explorer. Include a positive control: a query that SHOULD succeed, so that if the permission check is broken in the other direction I will see it. Show me the output of both.

A test where nothing fails has proved nothing. This is the single most valuable thing an assistant can do for you here.

5. Email

> Read `docs/EMAIL.md`. Supabase's built-in mailer sends two messages an hour for the whole project and that cannot be raised. Tell me whether my church needs an SMTP provider, and if so give me the exact dashboard steps for Brevo — including the IP-blocking setting, which applies to SMTP as well as to API keys. I will paste every credential myself. Do not ask me for a key and do not put one in a file.

6. Deploy

> Walk me through deploying to Vercel and tell me exactly which environment variables to set. Then tell me how to verify the deployment actually works, as a test I run rather than a thing you assert.

7. Make it prove the deployment, not describe it

> Give me a numbered list of things I click, on my phone, on mobile data with wifi off, that would show this deployment is genuinely working — including at least one that should FAIL, so I know the checks mean something. Do not tell me it works; tell me how I would find out that it does not.

Prompts worth keeping

Not a session — individual jobs, for when something is already running. Each one is written the way that actually gets a useful answer: it names the files, says what "done" looks like, and asks for evidence rather than assurance.

Diagnosing a failure

> Invitations stopped arriving. Do not guess. Query the Supabase auth logs, group the last 24 hours by hour and by status, and tell me the actual reason with the log lines that show it. If the answer is a rate limit, tell me the measured number rather than the documented one.

> This error appeared: `<paste the exact text>`. Before proposing a fix, tell me what state the system must be in for that message to be produced. Then tell me how to check whether it is in that state.

Changing the app safely

> I want `<change>`. Before writing any code, tell me which files it touches, what could break that I would not notice, and which existing test would catch a mistake. Then wait for me.

> Read `tests/` and tell me which of my behaviours are actually covered and which only look covered. I want the list of things that would pass while broken.

Making it your church's own

> Change every place the app says "Guide" and "Explorer" to `<our words>`. Find them all — including the tutorial, the emails and the policy page — and show me the list before you change anything.

> Rewrite `app/policy/page.tsx` to match our church's safeguarding policy, which is: `<paste yours>`. Keep the reading level and the structure. Do not add clause numbers or legal phrasing. Keep the emergency-services line first.

Reviewing what the assistant just did

> Read back the diff you just made and argue against it. What did you assume that I did not tell you? What would a reviewer object to? What did you not test?

> You said this is fixed. Show me the command I run and the output I should see. If you have not run it, say so plainly instead of describing what would happen.

The prompt to use when you are stuck and frustrated

> Stop proposing fixes. Write down, in order, what you actually know to be true from evidence you have seen this session, and separately what you are assuming. Then tell me which single assumption, if wrong, would explain everything.

Things to watch for

"It's working now." Ask how it knows. This project's own history has a fixed invitation system reported as fixed twice, while every link it produced still failed on arrival, because nobody clicked one.

Confident SQL against a schema it has not read. If it did not open

Security rules edited to make an error go away. "Permission denied" usually means the rules work. Weakening them to clear the error is how a church's private conversations end up readable by anyone holding a key that ships in every visitor's browser.

Any request for a secret. No assistant needs your `service_role` key, your Brevo key, or your database password. If one asks, say no and paste

---

If you do not have an AI assistant

Everything here can be done by hand. Part 7 of START-HERE.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/START-HERE.md) is the manual path with every command, and Part 8 is the same journey for someone who has never opened a terminal.

Email

Nothing to set up. Hope Beacon sends its invitations through Supabase's own email service, which every Supabase project has. No account with a third party, no API key to store, no sender address to verify. Clone this project, run the migrations, and invitations work.

> **Read this before your first busy day.** The built-in service sends **two messages an hour for the entire project**. That is fine for a church inviting one person at a time and will quietly strand you if you sit down to invite ten. Connecting an SMTP provider is a dashboard form, takes about five minutes, and raises it to thirty an hour.

How an invitation is actually sent

two things depending on whether the person already has an account:

Situation	What is sent
A new address	`inviteUserByEmail` — creates the account and sends "you have been invited"
Someone who never finished joining	`resetPasswordForEmail` — sends "set a password"

Both messages link to `/join`, which is the screen that finishes the sign-up either way. Pressing **Re-send** on a waiting invitation mints a fresh link and posts it again, with nothing to retype.

What this costs you

Being honest about the trade, because it matters at a certain size:

- **The wording is Supabase's, not your church's.** The message comes from your project's Auth email template. You can rewrite that template in the Supabase dashboard under **Authentication → Email Templates** and it will say whatever you like.
- **It sends two messages an hour. For the whole church.** Not per person — two, in total, per project, per hour. This number is measured, not estimated: the auth log of a real church shows at most two successes in any hour and `over_email_send_rate_limit` on everything after.
- **And one message per address per minute**, separately from the above.

What that means on the day you actually invite people

If your Director invites four people in one sitting, **two of those invitations are never sent.** The screen used to show the provider's raw refusal, which reads like a fault; it now says plainly that the hour's email is spent and hands over the join link for the person in front of you.

This limit cannot be raised while the built-in mailer is in use. It is not a setting in your dashboard. The only way to send more is to connect an email provider, below — which is why the section under it exists.

Telling the two refusals apart

Both come back as HTTP 429 and they mean different things:

What you see	What it is	What to do
"wait N seconds"	One per address per minute. The first message **was** accepted.	Wait, press Send once.
"used up its email for the hour"	The project's two-an-hour quota is spent. **Nothing was sent.**	Send the link on screen by hand, or connect a provider.

Why not a third-party provider by default

This project used to post invitations to Brevo, and it cost a full day of a church's time in a way worth recording.

Everything was configured correctly. The API key, the verified sender and the site URL were all present and all loaded — the function's own log confirmed it on every attempt. Every call was refused anyway, because the Brevo account had its **IP allow-list** switched on, and Supabase Edge Functions have no fixed IP address to add to it. Two consecutive attempts were rejected from two different servers. Invitations silently stopped because of a setting in a third party's dashboard that could not be seen from inside the app.

None of that can happen with Supabase's own mailer. It is sent by Supabase, from inside Supabase.

But note what the lesson actually was. It was not "Brevo is bad" — it was that a security toggle in a third party's dashboard could stop every invitation with no sign of it inside this app. Brevo is a perfectly good way to send this church's email, over SMTP, once that toggle is off. What changed is that the app now *names* the cause when it happens, instead of showing the raw refusal.

Adding a provider — sooner than "eventually", if you invite in batches

Two an hour is enough for a church adding one person after a Sabbath conversation. It is not enough for a launch, a training weekend, or a demonstration. If you have more than two people to invite in an hour, you need this section, and the good news is that it is a dashboard form rather than code.

The easy way: custom SMTP (no code at all)

Supabase's own Auth service will use your provider's SMTP server if you give it one, under **Project Settings → Authentication → SMTP Settings**. Everything in this app keeps working exactly as it does now — same function, same templates, same links, no key stored anywhere in this project — and the default quota rises from two an hour to **thirty new users an hour**, itself configurable under **Authentication → Rate Limits**.

You need four things from your provider, which every dashboard calls "SMTP credentials": host, port, username, password. Plus a **sender address the provider has verified as yours** — most refuse to send from one they have not, and the error rarely says so plainly.

Brevo, specifically — and the correction that matters

An earlier version of this page said the IP allow-list "was on the API-key path" and that whether it also covered SMTP was untested. **It covers SMTP.** Brevo's own documentation says the authorized-IP list is shared across API keys and SMTP keys, and an unrecognised address gets `525 5.7.1 Unauthorized IP address`.

So the thing that broke invitations here breaks the SMTP route too, and there is one way out of it:

> **Switch the blocking off. Do not add an address to the list.**
>
> The connection is made by your mail service's servers, not by the computer the Director is sitting at, and those addresses change between sends. Adding your own IP authorises the one machine that never connects.
>
> **Settings → Security → Authorized IPs → Deactivate blocking.**

Then, still in Brevo: **Transactional → Settings → SMTP relay → Generate a new SMTP key**. Copy the login (it looks like `something@smtp-brevo.com`) and the key. **An SMTP key is not an API key** — the API key will not authenticate here.

Into Supabase:

Field	Value
Host	`smtp-relay.brevo.com`
Port	`587`
Username	the SMTP login, `…@smtp-brevo.com`
Password	the SMTP **key**
Sender email	an address verified in Brevo
Sender name	your church's name

Last, raise the quota that was the whole problem: **Authentication → Rate Limits → emails sent per hour**, up from 2.

If invitations stop again after this, the Director now sees which of the four causes it was in plain words rather than the provider's raw text — including "that is IP blocking, and it has to be turned off".

Resend, if you prefer it

Resend works the same way and is written down here because it keeps being asked about. It is SMTP into the same Supabase box, so nothing in the app changes.

Field	Value
Host	`smtp.resend.com`
Port	`587`
Username	`resend` — the literal word, not your email
Password	an API key created in Resend, beginning `re_`
Sender email	an address on a domain you have verified in Resend

Two differences from Brevo worth knowing before you choose:

- **Resend has no authorized-IP feature**, so the trap that cost a day with Brevo does not exist here.
- **Resend will not send from a shared address at all.** Brevo lets you verify a single sender like `you@gmail.com` and start immediately; Resend requires a domain you control, verified with DNS records, before it sends anything to anybody but yourself. If you do not own a domain yet, that decides it.

The DNS records, whichever provider you pick

Both want the same three things on the domain, and all three are TXT records added at your registrar:

Record	What it does	What happens without it
SPF	Says which servers may send as your domain	Gmail marks the mail as suspicious or bins it
DKIM	Signs each message so it cannot be tampered with	Same, and some providers refuse to send at all
DMARC	Tells other mail servers what to do when the first two fail	Nothing breaks, but delivery is weaker and you get no reports

A reasonable first DMARC record is `v=DMARC1; p=none; rua=mailto:you@example`, on the host `_dmarc`. `p=none` means "watch, do not reject", which is where you want to start.

Give DNS an hour before deciding it has not worked. Most changes are live in minutes; some registrars are slower, and re-adding a record you already added is how people end up with two conflicting SPF lines, which is worse than none.

The other way: post to a provider's API from the function

Only if you want your church's own wording in the message body rather than Supabase's template. The send lives in **one place** — the block marked

a POST to your provider and nothing else in the app changes.

Whichever provider you choose:

- **Verify your sending address with them first.** Most refuse to send from an address they have not confirmed is yours, and the error rarely says so plainly.

- **Do not switch on an IP allow-list.** Edge Functions have no fixed address. This is the one that cost the day above.
- **Keep the key out of the browser.** It belongs in Edge Function secrets (**Edge Functions → Secrets**), or failing that in the `app_settings` table, which has row level security on with no policies and every grant revoked from `PUBLIC` — so only the service role can read it. Never in a `NEXT_PUBLIC_` variable.

When an invitation cannot be emailed

The Director gets the one-time link on screen with a **Copy link** button, and the invitation stays open. The account is real and the link works; only the postman is missing. Sending it by hand — WhatsApp, Messenger, in person — takes the person through exactly the same sign-up.

That path is deliberate. An email service having a bad afternoon should not stop a church inviting somebody standing in front of them.

Security

Read this before connecting Open Sentry Beacon to anything real. It is short on purpose, and it tries to be honest rather than reassuring.

Reporting a vulnerability

Do not open a public issue. Use GitHub's private Security Advisories (https://github.com/Klydo131/Open-Sentry-Beacon/security/advisories/new) on this repository, which is visible only to maintainers until a fix is published.

Include what an attacker can do stated as an outcome ("any visitor can read X"), the smallest steps that reproduce it, and the commit you tested. You will get an acknowledgement within **7 days**. There is no bounty programme — this is a small project maintained in spare time, and we would rather say so than imply a response time nobody can hold to.

What this app is, in security terms

As shipped, there is nothing to breach. No backend, no accounts, no network calls, no analytics. Every screen runs from a store in the browser, and the sample church is fiction. That is why you can hand it to anybody to try.

Three tests hold that promise rather than a paragraph in a README:

Check	What it refuses
`tests/no-backend.js`	A database dependency, an API route that stores data, any analytics or error-reporting SDK, any call to an external server.
`tests/no-secrets.js`	A credential of recognisable shape in any tracked file, a real `.env`, anything secret exposed under a browser-visible prefix, sample data at a domain that could reach a real inbox.
`tests/security-invariants.mjs`	A weakened Content-Security-Policy, a URL guard that stops blocking `javascript:`, CI that could be hijacked by a pull request.

Run them with `npm test`.

The moment that changes: connecting a backend

Everything above stops being the whole picture the day you point this at a real database. From then on **you own the security of what you connected**, and this is the part people get wrong.

Put authorisation in the database, not in the app

The screens in this app decide what to *show*. They cannot decide what somebody is *allowed to have*, because anybody can send a request without using your screens at all.

If your backend supports row-level rules, use them. A rule that lives with the data applies to every route into it — including a screen somebody adds next year without reading this file.

Never put a secret in this repository

Anything the browser can read, every visitor can read. That includes any key in a

backend adapter you write in `lib/backend/`.

Keys belong on a server you control. The browser talks to your server; your server holds the key. `tests/no-secrets.js` fails the build if a credential appears here, but it can only catch shapes it recognises — it is a safety net, not permission to be careless.

Rate limit on your server

Anything enforced in the browser can be skipped by opening the network tab. If an endpoint you add sends email, writes rows, or costs money, limit it server-side, key the limit on something the caller cannot change for free, and give anything that sends messages its own separate ceiling.

Decide what your leaders can see

This app is built so that leaders see counts and Guides see the relationship — a pastor cannot read a Guide's conversations, because there is no screen that shows them. If you connect a backend, that boundary is now yours to enforce in your data rules. It is easy to lose by accident and hard to explain afterwards.

Place suggestions go through your server

The meet-up place box suggests places from OpenStreetMap as somebody types. The browser never asks the map service itself: it asks `supabase/functions/places`, which checks the caller is signed in, approved and not suspended; limits them to the pace of a person typing; sends only the words and a town-level point; logs nothing; and holds no master key. The Content-Security-Policy therefore still allows the browser to talk to your backend and nothing else.

Nobody promotes themselves

A signed-in person cannot change their own role, approval, church or a guardian's consent. The database refuses it (`lock_privileged_profile_columns`, error 42501), whatever the screen sends. The one way through is claiming an unexpired invitation for exactly that church and role, while still unapproved. The app never asks, either: updating your own profile sends a fixed list of fields that has none of them. The sample app's role switcher changes a role only in the browser's own store, so a pastor can see every screen before adopting it; the live half never loads it. `tests/nobody-promotes-themselves.mjs` fails if any of that changes, including a later migration that drops the trigger.

An invitation is a way in for three days

An invitation creates the account and e-mails a temporary password, because a one-time link is spent by mail scanners and strands people. That password is now **a way in for three days and no longer** (seven from 29 September 2026, three since 3 October 2026). Since 3 October it is also three ordinary words and a two-digit number joined by dashes (`harbor-acorn-river-48`), about 1.5 billion possibilities, so guessing one in those three days is not practical even for somebody who knows the address it was sent to:

- **It runs out on the server, not in the app.** When the three days are up the database replaces it with a random password nobody knows and signs out every device that used it (`end_expired_temporary_passwords` , hourly via pg_cron). A letter found in an old inbox, a forwarded e-mail, or the Director's own screen is useless after that.
- **It can never touch a password somebody chose.** The database keeps the hash the invitation set and acts only while the account still has exactly that hash. A password changed by any door (the Password page, a reset e-mail, an admin) has a different hash and is left alone. The reminder flag is not trusted for this, because anybody signed in can clear it.
- **Choosing a password signs out every other device.** A new password always stopped the old one signing anybody in; now a phone or laptop already signed in with the letter is signed out too. The Password page also has **Sign out everywhere else**, for anybody who sees a sign-in they do not recognise.
- **An ended session is refused at once** (since 29 September 2026). Before, the pass a device already held kept working for up to an hour after its session was ended. Now the database asks, on every request, whether the session is still on record: the data API refuses an ended one with a 401, and the live socket and the file store stop at the same moment. The app hears the 401, asks the sign-in server once, and shows the front door. A signed-out visitor, the server's own requests, and a token with no session id are let through exactly as before.
- **A re-send cannot take over an account.** It is refused for anybody who has ever signed in (since 7 September 2026), so an invitation can only ever set the password of an account nobody has used.
- **None of it depends on the code being secret.** The table holding the clock has row-level security with no policies and no grants: no browser can read or change it. Only the invitation (the service role) starts a clock; nobody can run the expiry from a browser; a member can ask only when their own runs out. Knowing exactly how it works tells an attacker that an old letter is useless.

Accounts invited before 29 September are not on a clock unless the owner starts one: on that day 40 accounts were still flagged as using their e-mailed password, including the owner's own, and locking the owner out of their own account is not a choice to make for them. To give every one of them seven days (anybody who has ever asked for a reset e-mail is left out, because their flag may be stale), run once in the SQL editor:

```
select public.start_temporary_password(p.id)
from public.profiles p
join auth.users u on u.id = p.id
where p.password_is_temporary and u.recovery_sent_at is null;
```

was run on the live database in a transaction that was thrown away, with two made-up accounts, before it was applied: the letter's password stopped working and its sessions ended; the account that had chosen its own was untouched.

Two-step sign-in

Since 13 October 2026 anybody can add a second step to their sign-in: a six-digit code from an authenticator app on their phone (Settings, Password, **Two-step sign-in**). It is recommended on that page to Directors and Executive Directors, who can see the church's list of members. Supabase does the cryptography; the app only asks it (`lib/live/two-step.ts`).

- **The lock is in the database, not the screen.** A password alone still produces a valid token, marked `aal1`; password and code produce `aal2`. For anybody with a confirmed authenticator app, the data API's check before every request refuses an `aal1` token (HTTP 403), and the live socket and the file store ask the same question. Somebody who learns the password and calls the data API directly gets nothing (`20261013140000_a_second_step_once_set_up_is_required`).
- **It changes nothing for anybody who has not turned it on.** A person with no authenticator app, or one they started adding and never confirmed with a code, is let in on the password exactly as before. The signed-out visitor and the server's own requests are untouched.
- **The invite function asks too.** It reads with the service role key, so the database's refusal never reaches it; it refuses a password-only token from a Director who has two-step sign-in on.
- **A lost phone.** The person cannot finish signing in until a custodian removes their authenticator app. **Confirm who is asking first**, in person or by a phone number you already have: removing it is exactly what somebody who stole the password would ask for. Then, in the SQL editor:

``` `sql delete from auth.mfa_factors where user_id = (select id from auth.users where email = 'their address');` ```

They sign in with their password and can set it up again. (Written from the sign-in service's table shape; not yet run on a live project.)

- **Supabase must allow it.** Authentication, Multi-Factor, authenticator app (TOTP) is on by default. If it has been switched off, setting up says so and where to switch it on.

invented people in every fresh install; `tests/two-step-sign-in.mjs` runs the app's half against a stand-in sign-in server.

Files people upload

The live app keeps every uploaded file in **one private storage bucket**, and a rule per folder decides who may put a file there and who may open it. Nothing in that bucket is public: a file is opened through a signed address that expires after an hour.
```

Folder	Who may upload	Who may open
`<pairing>/`	Either person in that conversation	The same two
`avatars/<person>/`	That person	People in their church
`library/<person>/` (Resources)	That person, if a Guide, Explorer or leader and not stopped from sharing	That person, and whoever can read a resource pointing at the file
`lessons/<person>/` (study handouts)	That person, if they may write studies	Whoever can read the study
`reports/<person>/` (safeguarding evidence)	That person	The church's leadership

Four rules apply across all of them:

- **Every file is at most 10 MB**, and only pictures, PDFs, office documents, audio and text are accepted. There is no video and no SVG, which can carry script.
- **Each person can keep up to 300 files or 200 MB** in their Resources and handouts. That bounds what one stolen password, or one script driving an account, can cost the church.
- **A suspended account can neither upload nor open anything.**
- **A photo loses the location its camera recorded** before it is sent. The exception is safeguarding evidence, which is kept exactly as sent.

These rules live in `supabase/migrations/`, and

fail if they are loosened.

A folder does not say whose church it is. The rules above find a file's church from the person's id at the start of its folder, through

called on its own, and it told any signed-in member which church any person was in. It now answers only with a church the caller can already see into or leads. Checked on the live database before it was applied, for every account against every person's folder: all 1,849 pairs got the same answer from both storage tests as before, and the 42 answers that had crossed from one church into another became none. `tests/a-folder-does-not-say-whose-church.mjs` holds it, including that every rule keeps using the helper only inside those two tests.

A drawing carries the picture and nothing else. A drawing made on a study (Excalidraw, since 28 September 2026) is saved as a PNG with the drawing itself in one text chunk, so it can be reopened and changed. It goes through the same upload and the same 10 MB limit as any handout. Every other chunk a PNG can carry -- a camera's EXIF, where a location lives, a colour profile naming a device, free text -- is cut out before it is sent (`lib/drawing-file.ts`), so a photo renamed to look like a drawing still arrives with nothing but its pixels. Photos cannot be put inside a drawing at all: a photo there would skip the step that removes its location, so it is added as a handout instead.

The drawing board talks to nobody else. Excalidraw is loaded only when somebody presses *Draw a picture*. Its fonts are copied out of the installed package into `public/excalidraw/` at build time and served by the app itself; its fallback to a public CDN is removed before the browser sees it, and the Content-Security-Policy was not widened

for any of it. Its AI tools, web embeds, online library and file export are switched off.

any request leaves the app or anything is refused by the policy.

Settings that live outside the code

Some protections are switches in Supabase and GitHub, which no file in this repository can turn on. Check them once when you set up, and again after anybody new is given access. Listed after the audit of 3 October 2026.

Where	Set it to	Why
Supabase → Authentication → "Allow new users to sign up"	**Off**	The app is invitation-only, and invitations are made through the service role, so nothing breaks. Left on, the public key alone can create accounts.
Supabase → Authentication → email code expiry	**Under an hour**	A sign-in code that lasts longer is longer to steal.
Supabase → Authentication → Multi-Factor → authenticator app (TOTP)	**Enabled** (the default)	Two-step sign-in in the app uses it. Switched off, nobody can set it up.
GitHub → Settings → Environments → `backup`	Allow `main` only, you as **required reviewer**, `SUPABASE_DB_URL` and `BACKUP_PASSPHRASE` as its secrets	The database address can read everything. As a repository secret, any workflow on any branch can read it, so anybody who can push, or an AI tool working with that access, could copy it out. `.github/workflows/backup.yml` explains.
GitHub → Settings → Code security	**Private vulnerability reporting, secret scanning and push protection on**	The "report a vulnerability" link above depends on the first; the others stop a key from being pushed by mistake.
GitHub → Settings → Branches (or Rules) for `main`	**Require review from Code Owners**	Then a pull request that touches the files in `.github/CODEOWNERS` (database rules, workflows, server routes, the AI brief) waits for your review.
Any AI tool's connection to your database or hosting	**Ask every time** for running SQL, applying migrations and deploying	The prompt is what stands between an AI session and your live data. `AGENTS.md` section 0.3 says why an AI tool must not act on instructions it merely read.

Search engines

Your deployment is invisible to search by default, and a church should leave it that way. A church Beacon holds real people's names and conversations, and a shared deep link that gets indexed is the cheapest possible leak — nobody has to break anything, they only have to search.

Three signals say so together: a `<meta name="robots">` tag on every page,

variable, so they cannot end up disagreeing with each other:

```
BEACON_PUBLIC_SITE=1   # opt IN to being findable. Unset, the default, means no.
```

Set it only on a deployment with no real people in it — a public demo or a showcase, where being unfindable is the bug. Do not set it on a church's Beacon.

What this is not: `robots` directives are a request. The large search engines honour them and anything that does not care ignores them. They keep your app out of Google; they are not access control. If a page must not be read by a stranger, it needs a sign-in, not a header.

Sample data

The sample church is fiction and must stay fiction. Names use reserved domains (`.example` , `.test`) that can never resolve to a real inbox, and a test enforces it. **Never commit real people's details**, not even briefly, not even in a branch — a public repository is indexed within minutes and git remembers.

Supported versions